SALUNIC

Sistema de Salud y Bienestar

GUIA DE ESTUDIO FINAL - EXPLICATIVA

App Tkinter + Modulo Consola (Integrados)

Estructura requerida por el profesor:

Entidades/ + Datos/ + Funciones/ + screens/ (app) + consola/

Temas: DataClass | Cola con Prioridad (heapq) | Busqueda Lineal | Archivo JSON

Actualizado: 06/11/2026 | *Codigo fuente: GitHub (salunic-app)*

Objetivo de este PDF: que puedas estudiar y entender TODO el proyecto (estructura, reglas, conexion entre partes, codigo clave y capturas del codigo real) para explicarlo y defenderlo con seguridad.

INDICE

1. Que pide el profesor y que cumple el proyecto
2. Estructura FINAL de carpetas (arbol) y para que sirve cada una
3. Reglas de acceso (MUY IMPORTANTE)
4. Entidades/ - Las 4 DataClass (modelos de datos)
5. Datos/ - Listas en memoria + datos iniciales + JSON (persistencia)
6. Funciones/ - Una por cada DataClass + utilidades (logica)
7. Cola con Prioridad (heapq): como funciona y por que adulto mayor va primero
8. Busqueda Lineal: como funciona y donde se usa
9. Archivo de Datos JSON: guardar y cargar (asdict / reconstruccion)
10. App Tkinter (app/): como usa solo Funciones y Entidades
11. Modulo Consola (consola/): como usa Entidades, Datos y Funciones
12. Conexion App <-> Consola <-> Datos compartidos (flujo completo)
13. Archivos clave explicados (que hace cada uno)
14. Como ejecutar y probar
15. Tips para exponer y defender (guion + respuestas tipicas)
16. Checklist final
17. Capturas del codigo real (evidencias para estudiar)

1. Que pide el profesor y que cumple el proyecto

El profesor indico una estructura y reglas claras para organizar el codigo, ademas de los temas de programacion solicitados. SALUNIC cumple con lo siguiente:

Temas / requisitos funcionales

- Lista de DataClass (4 clases de entidad).
- Uso de Cola con PRIORIDAD (no FIFO puro): el adulto mayor se atiende antes aunque llegue despues.
- Busqueda lineal (recorrer elemento por elemento).
- Archivo de datos (persistencia en JSON).
- App Tkinter y modulo por consola funcionando e integrados con la misma logica/datos.

Estructura requerida

- Entidades/: DataClass (modelos).
- Datos/ (o Datos/Data): listas con datos guardados + carga/guardado.
- Funciones/: un archivo por cada DataClass (y utilidades relacionadas).
- screens/: pantallas de la app (Tkinter).

Idea central

Separar responsabilidades: Entidades define "que es" cada dato, Datos guarda "los datos", Funciones define "que se puede hacer" con esos datos, y las interfaces (screens de app y consola) usan esas funciones. Asi el codigo es ordenado, reutilizable y facil de defender.

2. Estructura FINAL de carpetas (arbol)

```
C:\salunic-app\
|-- Entidades/                # DataClass (4)
|   |-- __init__.py
|   |-- usuario.py            # Usuario
|   |-- cita_medica.py        # CitaMedica
|   |-- medicamento.py        # Medicamento
|   `-- almacen_medicamento.py # AlmacenMedicamento
|
|-- Datos/                    # Listas + persistencia JSON
|   |-- __init__.py
|   `-- datos.py              # listas, datos iniciales, cargar/guardar JSON
|
|-- Funciones/                # Logica (1 por dataclass) + utilidades
|   |-- __init__.py
|   |-- usuario.py            # CRUD/ops de Usuario
|   |-- cita_medica.py        # CRUD/ops de CitaMedica
|   |-- medicamento.py        # CRUD/ops de Medicamento
|   |-- almacen_medicamento.py # CRUD/ops de Almacen/Stock
|   |-- sistema.py            # funciones de alto nivel para la app
|   |-- busqueda_lineal.py    # busqueda lineal
|   |-- cola_atencion.py      # cola con prioridad (heapq)
|   `-- validaciones.py       # validaciones/reutilizacion
|
|-- app/                      # App grafica Tkinter
|   |-- main.py
|   |-- data/
|   `-- screens/              # UI (usa Funciones y Entidades)
|
|-- consola/                  # Modulo por consola (usa Entidades, Datos, Funciones)
|   `-- main.py
|
|-- core/                     # Compatibilidad / version previa (no requerida)
|-- docs/                     # Documentacion y PDFs
|-- scripts/                  # Scripts auxiliares
|-- salunic_datos.json        # Archivo de datos compartido (persistencia)
`-- README / .bat
```

Nota: "core/" se conserva SOLO por compatibilidad y delega a Entidades/Datos/Funciones; no es la estructura principal. La estructura que se debe mostrar/defender es Entidades + Datos + Funciones, con app/screens y consola consumiendo esa logica.

Archivos de apoyo actuales: app/data/users_data.py (adaptador de compatibilidad de la app) y scripts/ (generacion de capturas y documentos). No forman parte de la logica principal.

3. Reglas de acceso (MUY IMPORTANTE)

Estas reglas evitan el desorden y demuestran buena arquitectura:

Regla A - screens/ (interfaz grafica)

Los archivos dentro de screens/ SOLO pueden usar: Funciones y Entidades.

NO deben acceder directamente a Datos/ (ni leer/modificar listas desde la UI). La UI pide acciones a Funciones (ej: sistema.autenticar_usuario, sistema.agregar_cita, sistema.agregar_a_cola).

Regla B - Funciones/

Los archivos de Funciones SOLO pueden usar: Datos y Entidades.

Aqui vive la logica: validaciones, CRUD, busqueda, cola, y operaciones que modifican las listas de Datos.

Regla C - consola/

El modulo consola usa Datos y Funciones (las Entidades se crean/usan dentro de Funciones) para cumplir lo requerido y compartir la misma logica. Importa menus/acciones desde Funciones y puede leer listas de Datos para flujos interactivos (ej: encolar una cita existente).

En el codigo actual consola/main.py importa: Datos (listas y cargar/guardar), Funciones.usuario/cita_medica/medicamento/almacen_medicamento y Funciones.cola_atencion.

Por que importa

Si la UI tocara Datos directamente, se duplicaria logica y seria mas dificil mantener. Centralizar en Funciones permite que app y consola se comporten igual y usen el mismo archivo JSON.

4. Entidades/ - Las 4 DataClass

Una DataClass (@dataclass) es una clase de Python que simplifica crear "modelos" para guardar datos (genera automaticamente __init__, representacion, etc.). Cada entidad representa algo real del sistema.

```
# Entidades/usuario.py
from dataclasses import dataclass

@dataclass
class Usuario:
    id: int
    nombre: str
    email: str
    password: str
    telefono: str = "+505 00000000"
```

Explicacion: Usuario guarda identificacion y credenciales/contacto. Se usa para login/registro y para relacionar citas (paciente/medico por id).

```
# Entidades/cita_medica.py
@dataclass
class CitaMedica:
    id: int
    paciente_id: int
    medico_id: int
    especialidad: str
    fecha: str   # DD/MM/AAAA
    hora: str    # HH:MM
    lugar: str = "Hospital"
```

Explicacion: CitaMedica conecta un paciente con un medico y guarda cuando/donde es la cita. Las citas son las que se pueden enviar a la cola de atencion.

```
# Entidades/medicamento.py / almacen_medicamento.py
@dataclass
class Medicamento:
    id: int; nombre: str; dosis: str; frecuencia: str; presentacion: str = "Tabletas"

@dataclass
class AlmacenMedicamento:
    id: int; medicamento_id: int; cantidad: int; fecha_vencimiento: str
```

Explicacion: Medicamento describe el producto; AlmacenMedicamento representa el stock/inventario (cantidad y vencimiento) asociado a un medicamento_id. Esto separa catalogo vs inventario.

Las 4 DataClass pedidas estan presentes: Usuario, CitaMedica, Medicamento, AlmacenMedicamento. Se exportan desde Entidades/__init__.py para importarlas facil: from Entidades import Usuario, ...

5. Datos/ - Listas en memoria + JSON

Datos/datos.py cumple el rol de "almacen de datos" del programa:

1) Define listas en memoria (mientras corre el programa): usuarios, citas, medicamentos, almacen.

2) Define datos iniciales de ejemplo para pruebas.

3) Implementa persistencia con Archivo JSON: cargar_datos() y guardar_datos().

```
# Datos/datos.py (idea)
from dataclasses import asdict
from Entidades import Usuario, CitaMedica, Medicamento, AlmacenMedicamento
import json, os

usuarios = []
citas = []
medicamentos = []
almacen = []

ARCHIVO_DATOS = ... "salunic_datos.json" (ruta en la raiz del proyecto)

def guardar_datos():
    data = {"usuarios":[asdict(u) for u in usuarios], ...}
    json.dump(data, f, ensure_ascii=False, indent=4)

def cargar_datos():
    data = json.load(f)  # utf-8-sig para tolerar BOM
    usuarios.clear(); ...
    usuarios.append(Usuario(**u))  # reconstruccion desde dict
```

Puntos clave para explicar:

- "Listas de las dataclass": cada lista contiene objetos de su DataClass correspondiente.

- Se guarda en JSON con asdict() (convierte el objeto DataClass a diccionario).

- Se carga reconstruyendo objetos con Clase(**diccionario).

- El JSON se guarda en la raiz (salunic_datos.json) para que app y consola compartan los mismos datos.

6. Funciones/ - Una por cada DataClass + utilidades

La carpeta Funciones contiene la logica del sistema. Cumple la regla: usa solo Datos y Entidades.

Un archivo por cada DataClass (requisito)

- Funciones/usuario.py -> operaciones de Usuario (registrar, buscar, listar, actualizar, eliminar, contar).
- Funciones/cita_medica.py -> operaciones de CitaMedica.
- Funciones/medicamento.py -> operaciones de Medicamento.
- Funciones/almacen_medicamento.py -> operaciones de AlmacenMedicamento (stock).

Utilidades / modulos de apoyo

- Funciones/busqueda_lineal.py -> implementa busqueda lineal (requisito).
- Funciones/cola_atencion.py -> cola con prioridad usando heapq (requisito).
- Funciones/validaciones.py -> valida entradas (nombre, email, fecha, hora, etc.).
- Funciones/sistema.py -> funciones de alto nivel para la app (iniciar/guardar, autenticar, registrar, agregar cita, cola) respetando reglas desde la UI.

```
# Ejemplo (idea): Funciones/usuario.py
from Entidades import Usuario
from Datos import usuarios
from .validaciones import ...

def buscar_usuario(email):
    for u in usuarios:           # acceso a Datos (lista)
        if u.email == email:
            return u
    return None
```

Para la defensa: "Cada DataClass tiene su archivo de funciones, y las funciones operan sobre las listas de Datos usando las Entidades".

7. Cola con Prioridad (heapq)

Requisito clave: NO es una cola FIFO pura. Si llega un adulto mayor despues, debe atenderse antes que un paciente normal que llego antes (prioridad).

Como se implementa

Se usa el modulo heapq de Python (cola de prioridad / heap). Guardamos elementos como tuplas: (prioridad, turno_global, item). heapq siempre extrae el elemento con MENOR valor en la primera posicion.

```
import heapq

# encolar
if es_adulto_mayor:
    prioridad = 0   # MAYOR prioridad
    tipo = "ADULTO MAYOR (PRIORITARIO)"
else:
    prioridad = 1   # NORMAL

heapq.heappush(self.cola, (prioridad, self.turno_global, item))

# desencolar (atiende al de mayor prioridad)
prioridad, turno, atendido = heapq.heappop(self.cola)
```

Por que funciona

- prioridad=0 (adulto mayor) es menor que prioridad=1 (normal), por eso heapq lo saca primero.

- turno_global es un contador incremental: desempata cuando dos pacientes tienen la misma prioridad (conserva orden de llegada dentro del mismo nivel).

- Resultado: adulto mayor primero, sin importar el orden de llegada frente a normales.

Donde se ve

- Consola: menu "Cola de Atencion con Prioridad" (agregar, atender, mostrar).

- App: pantalla Citas (agregar a cola) y pantalla "Panel de Atencion" (visualiza cola y atiende siguiente).

8. Busqueda Lineal

La busqueda lineal recorre una lista elemento por elemento, comparando hasta encontrar el/los que coinciden. Es simple y clara para explicar.

```
# Funciones/busqueda_lineal.py (idea)
def busqueda_lineal(lista, criterio, atributo):
    resultados = []
    for elemento in lista:                # recorre uno por uno
        valor = getattr(elemento, atributo, None)
        if str(valor).lower() == str(criterio).lower():
            resultados.append(elemento)
    return resultados

def busqueda_lineal_por_email(usuarios, email):
    for u in usuarios:
        if u.email.lower() == email.lower():
            return u
    return None
```

Donde se usa

- Login de la app: autentica buscando el usuario por email con busqueda lineal (via Funciones.sistema / busqueda_lineal_por_email) y comparando password.

- Puede usarse tambien para filtrar citas/medicamentos por criterio (segun funciones disponibles).

Para la defensa: "Uso un ciclo for, reviso cada elemento y comparo; si coincide, lo retorno/agrego. Eso es busqueda lineal".

9. Archivo de Datos JSON (persistencia)

La persistencia permite que los datos no se pierdan al cerrar el programa. Se usa un archivo JSON compartido: salunic_datos.json (en la raiz del proyecto).

Guardar (serializar)

- Tomamos las listas de DataClass (usuarios, citas, medicamentos, almacen).
- Convertimos cada objeto a diccionario con asdict(obj).
- Escribimos el JSON con json.dump(..., ensure_ascii=False, indent=4).

Cargar (deserializar)

- Leemos el JSON con json.load (usando utf-8-sig por compatibilidad).
- Limpiamos las listas actuales (clear) para evitar duplicados.
- Reconstruimos objetos DataClass: Usuario(**u), CitaMedica(**c), etc., y los agregamos a las listas.

Integracion

Como app y consola usan la misma estructura de Datos/Funciones y el mismo archivo JSON en la raiz, ambos pueden trabajar sobre los mismos datos guardados.

10. App Tkinter (app/) - usa solo Funciones y Entidades

La app grafica esta en app/ (Tkinter). Las pantallas importantes fueron ajustadas para cumplir la regla del profesor.

Regla aplicada en screens/

app/screens/* importa y usa Funciones (ej: Funciones.sistema) y puede usar Entidades si es necesario; NO accede directamente a Datos/.

Ejemplos reales en el proyecto

- app/screens/login_screen.py: llama sistema.autenticar_usuario(email, password). La busqueda lineal ocurre dentro de Funciones.

- app/screens/registro_screen.py: llama sistema.registrar_usuario_app(...) para crear el usuario (DataClass) y guardar.

- app/screens/citas_screen.py: obtiene/agrega citas via sistema.obtener_citas()/agregar_cita() y encola con sistema.agregar_a_cola(..., es_adulto_mayor).

- app/screens/atencion_screen.py: "Panel de Atencion" muestra la cola (sistema.obtener_cola()) y atiende con sistema.atender_siguiente() usando la cola con prioridad.

app/main.py

Precarga datos con Funciones.sistema.iniciar_sistema() y gestiona navegacion entre pantallas. La UI es visual (Tkinter), pero la logica de negocio viene de Funciones/Datos/Entidades.

11. Modulo Consola (consola/) - alineado a la estructura

El modulo por consola ofrece menus para gestionar Usuarios, Citas, Medicamentos, Almacen, Cola de Atencion y Archivo de Datos.

Como cumple la estructura requerida

consola/main.py importa desde:

- Datos: listas (usuarios, citas, medicamentos, almacen) y Datos.datos (cargar_datos, guardar_datos, cargar_datos_iniciales).

- Funciones por dataclass: usuario, cita_medica, medicamento, almacen_medicamento (CRUD).

- Funciones.cola_atencion (cola_atencion con heapq).

Las Entidades se usan dentro de Funciones (por ejemplo, Funciones/usuario.py crea Usuario). La consola no duplica esa logica.

Que demuestra la consola

- CRUD por entidad (a traves de Funciones).

- Cola con prioridad: permite marcar adulto mayor y atiende por prioridad.

- Archivo JSON: guardar/cargar desde el menu.

- Comparte el mismo salunic_datos.json que la app, por lo que la integracion es real.

12. Conexion App <-> Consola <-> Datos compartidos

Este es el punto mas importante para defender la integracion:

Flujo de datos y logica

1) Entidades definen la forma de los objetos (DataClass).

2) Datos mantiene las listas (usuarios, citas, etc.) y el JSON compartido (salunic_datos.json).

3) Funciones implementa la logica (validar, buscar lineal, encolar con prioridad, CRUD, guardar/cargar).

4) La App (screens) llama a Funciones.sistema / Funciones.* sin tocar Datos directamente.

5) La Consola llama a Funciones y usa Datos segun el flujo interactivo del menu.

6) Ambos leen/escriben el MISMO archivo JSON -> los cambios persisten y se comparten.

Ejemplo de flujo (login en app)

Usuario ingresa email/password en pantalla -> login_screen llama sistema.autenticar_usuario -> Funciones usa busqueda_lineal_por_email sobre Datos.usuarios -> retorna el Usuario (DataClass) si coincide.

Ejemplo de flujo (cola con prioridad)

En Citas (app) o en Consola se indica si es adulto mayor -> se llama encolar(..., es_adulto_mayor=True/False) -> heapq ordena por (prioridad, turno) -> "Atender siguiente" siempre saca primero prioridad 0 (adulto mayor). El Panel de Atencion de la app lo muestra visualmente.

13. Archivos clave explicados

Entidades/

- usuario.py, cita_medica.py, medicamento.py, almacen_medicamento.py: definen las 4 DataClass. Son modelos simples y claros.

Datos/datos.py

- Listas globales del sistema + cargar_datos_iniciales() + guardar_datos()/cargar_datos() con JSON. Punto unico de persistencia compartida.

Funciones/ (por dataclass)

- usuario.py, cita_medica.py, medicamento.py, almacen_medicamento.py: CRUD y operaciones basicas. Usan Datos (listas) y Entidades (crear objetos).

Funciones/sistema.py

- Capa de servicios para la app: iniciar_sistema, autenticar_usuario (busqueda lineal), registrar_usuario_app, agregar_cita, agregar_a_cola/atender_siguiente/obtener_cola. Facilita que screens no toquen Datos.

Funciones/busqueda_lineal.py

- Implementa recorrido elemento por elemento. Incluye helpers como busqueda por email.

Funciones/cola_atencion.py

- Clase ColaAtencion con heapq, prioridad adulto mayor, turno_global, encolar/desencolar/mostrar_cola. Instancia global cola_atencion.

app/screens/atencion_screen.py

- Panel visual de atencion: muestra "EN ATENCION" y "COLA DE ESPERA", y boton para atender siguiente (demostracion fuerte de la cola con prioridad).

consola/main.py

- Menu interactivo que integra todas las funciones requeridas (DataClass via modelo, cola, busqueda lineal en login/logica relacionada, JSON) usando la estructura Entidades/Datos/Funciones.

14. Como ejecutar y probar

Requisitos

- Python 3 instalado. Tkinter suele venir incluido. (Para generar PDFs se uso fpdf en scripts; no es necesario para correr la app/consola.)

Ejecutar app (Tkinter)

Desde la carpeta del proyecto: ejecutar el lanzador .bat de la app o: py app/main.py (segun configuracion del entorno).

Ejecutar consola

Ejecutar run_consola.bat o: py consola/main.py

Pruebas rapidas sugeridas

1) Consola -> cargar/guardar JSON y listar entidades (verifica Datos/JSON).

2) Consola -> Cola de atencion: agregar normal y adulto mayor; atender y confirmar que el adulto mayor sale primero (verifica heapq/prioridad).

3) App -> Login/Registro (verifica busqueda lineal/autenticacion y uso de Funciones).

4) App -> Citas: agregar cita y agregar a cola con prioridad.

5) App -> Panel de Atencion: ver cola ordenada y atender siguiente (demostracion visual).

6) Verificar que salunic_datos.json se actualiza en la raiz del proyecto.

15. Tips para exponer y defender (guion)

Guion corto (1-2 minutos)

"SALUNIC esta organizado con la estructura que pidio el profesor: Entidades (las 4 DataClass), Datos (listas y archivo JSON compartido) y Funciones (una por cada dataclass y utilidades). Las pantallas de la app solo usan Funciones y Entidades, y las Funciones solo usan Datos y Entidades. La consola tambien usa la misma estructura, por eso app y consola comparten la misma logica y el mismo salunic_datos.json."

Puntos que debes mencionar si o si

- 4 DataClass: Usuario, CitaMedica, Medicamento, AlmacenMedicamento.

- Cola con prioridad con heapq: adulto mayor prioridad 0, normal 1, turno_global desempata; heapq saca el menor -> adulto mayor primero.

- Busqueda lineal: for elemento por elemento; usada en autenticacion/busquedas.

- Archivo JSON: asdict para guardar, Clase(**dict) para cargar; archivo compartido.

- Reglas de acceso: screens -> Funciones/Entidades; Funciones -> Datos/Entidades.

Respuestas tipicas

P: "Por que no FIFO?" R: "Porque el requisito exige prioridad para adulto mayor; uso heapq con (prioridad, turno) para que se atienda antes aunque llegue despues."

P: "Como se conectan app y consola?" R: "Comparten Entidades, Datos y Funciones, y el mismo JSON en la raiz; la logica no esta duplicada."

P: "Donde esta la busqueda lineal?" R: "En Funciones/busqueda_lineal.py y se aplica en la autenticacion/consultas recorriendo la lista completa."

P: "Que hace asdict?" R: "Convierte un objeto DataClass en diccionario para poder guardarlo en JSON."

16. Checklist final

[ ] Puedo mostrar las carpetas Entidades/, Datos/, Funciones/, app/screens/, consola/

[ ] Puedo explicar las 4 DataClass y donde estan

[ ] Puedo explicar que screens solo usan Funciones y Entidades

[ ] Puedo explicar que Funciones solo usan Datos y Entidades

[ ] Puedo mostrar/explicar la cola con prioridad (heapq, adulto mayor primero)

[ ] Puedo explicar la busqueda lineal y donde se usa

[ ] Puedo explicar guardar/cargar JSON (asdict y reconstruccion)

[ ] Puedo demostrar la app (login/citas/panel de atencion)

[ ] Puedo demostrar la consola (menus, cola, JSON)

[ ] Puedo explicar que app y consola comparten salunic_datos.json

Conclusion: si entiendes la separacion Entidades-Datos-Funciones, las reglas de acceso, y los 4 temas (DataClass, cola con prioridad, busqueda lineal, JSON), puedes explicar y defender el proyecto completo con confianza.

17. Capturas del codigo real (evidencias)

Estas capturas corresponden al codigo actual del proyecto y sirven para estudiar/ensayar la defensa mostrando archivos y lineas concretas. Se generan con: py scripts/generar_capturas_codigo.py

Carpeta: docs/capturas/codigo/

Codigo 1 - Entidades (4 DataClass con @dataclass)

1) Módulos y DataClass

Entidades (4 modelos con @dataclass) | docs/capturas/codigo/_tmp_ent.txt (líneas 1-999)

```python
# Entidades/usuario.py
from dataclasses import dataclass


@dataclass
class Usuario:
    id: int
    nombre: str
    email: str
    password: str
    telefono: str = "+505 00000000"

# Entidades/cita_medica.py
from dataclasses import dataclass


@dataclass
class CitaMedica:
    id: int
    paciente_id: int
    medico_id: int
    especialidad: str
    fecha: str        # formato DD/MM/AAAA
    hora: str         # formato HH:MM
    lugar: str = "Hospital"

# Entidades/medicamento.py
from dataclasses import dataclass


@dataclass
class Medicamento:
    id: int
    nombre: str
    dosis: str
    frecuencia: str
    presentacion: str = "Tabletas"

# Entidades/almacen_medicamento.py
from dataclasses import dataclass


@dataclass
class AlmacenMedicamento:
    id: int
    medicamento_id: int
    cantidad: int
    fecha_vencimiento: str  # formato DD/MM/AAAA
```

Codigo 2 - Datos y persistencia JSON (asdict, cargar/guardar, excepciones)

2) Datos y Persistencia JSON

Listas, asdict() y cargar/guardar (manejo de excepciones) | Datos/datos.py (líneas 1-78)

```python
import json
import os
from dataclasses import asdict

from Entidades import Usuario, CitaMedica, Medicamento, AlmacenMedicamento

# Listas propias de cada entidad (se guardan en memoria mientras corre el programa)
usuarios = []
citas = []
medicamentos = []
almacen = []


# Datos iniciales para pruebas
usuarios_iniciales = [
    Usuario(id=1, nombre="Ana Lopez", email="ana.lopez@salunic.com", password="123456", telefono="+505 88888888"),
    Usuario(id=2, nombre="Carlos Ruiz", email="carlos.ruiz@salunic.com", password="123456", telefono="+505 77777777"),
    Usuario(id=3, nombre="Maria Gonzalez", email="maria.gonzalez@salunic.com", password="123456", telefono="+505 66666666"),
]

citas_iniciales = [
    CitaMedica(id=101, paciente_id=1, medico_id=201, especialidad="Medicina General", fecha="30/10/2025", hora="09:00", lugar="Consultorio 1"),
    CitaMedica(id=102, paciente_id=2, medico_id=202, especialidad="Pediatria", fecha="30/10/2025", hora="10:30", lugar="Consultorio 2"),
    CitaMedica(id=103, paciente_id=3, medico_id=201, especialidad="Medicina General", fecha="31/10/2025", hora="14:00", lugar="Consultorio 1"),
]

medicamentos_iniciales = [
    Medicamento(id=301, nombre="Paracetamol", dosis="500mg", frecuencia="Cada 8 horas", presentacion="Tabletas"),
    Medicamento(id=302, nombre="Amoxicilina", dosis="500mg", frecuencia="Cada 12 horas", presentacion="Capsulas"),
    Medicamento(id=303, nombre="Ibuprofeno", dosis="400mg", frecuencia="Cada 8 horas", presentacion="Tabletas"),
]

inventario_inicial = [
    AlmacenMedicamento(id=401, medicamento_id=301, cantidad=150, fecha_vencimiento="12/2026"),
    AlmacenMedicamento(id=402, medicamento_id=302, cantidad=80, fecha_vencimiento="08/2026"),
    AlmacenMedicamento(id=403, medicamento_id=303, cantidad=200, fecha_vencimiento="10/2026"),
]

# Ruta absoluta al archivo de datos (en la raíz del proyecto) para consistencia
ARCHIVO_DATOS = os.path.join(os.path.dirname(os.path.dirname(os.path.abspath(__file__))), "salunic_datos.json")


def cargar_datos_iniciales():
    """Carga datos de ejemplo si las listas estan vacias."""
    if not usuarios:
        usuarios.extend(usuarios_iniciales)
    if not citas:
        citas.extend(citas_iniciales)
    if not medicamentos:
        medicamentos.extend(medicamentos_iniciales)
    if not almacen:
        almacen.extend(inventario_inicial)


def guardar_datos():
    """Guarda todas las listas (DataClass) en un archivo JSON"""
    try:
        data = {
            "usuarios": [asdict(u) for u in usuarios],
            "citas": [asdict(c) for c in citas],
            "medicamentos": [asdict(m) for m in medicamentos],
            "almacen": [asdict(a) for a in almacen]
        }
        with open(ARCHIVO_DATOS, "w", encoding="utf-8") as f:
            json.dump(data, f, ensure_ascii=False, indent=4)
        print(f"\n[OK] Datos guardados correctamente en: {ARCHIVO_DATOS}")
    except Exception as e:
        print(f"\n[ERROR] No se pudieron guardar los datos: {e}")


def cargar_datos():
    """Carga datos desde archivo JSON hacia las listas de DataClass"""
    if not os.path.exists(ARCHIVO_DATOS):
        print("\n[INFO] No existe archivo de datos. Se trabajara con datos en memoria.")
        return
    try:
        with open(ARCHIVO_DATOS, "r", encoding="utf-8-sig") as f:
            data = json.load(f)
```

Codigo 3 - Validaciones (ValueError) y manejo de excepciones

3) Validación y Manejo de Excepciones

Validadores (ValueError) y pedir_campo() con try/except | Funciones/validaciones.py (líneas 1-82)

```python
import re
from datetime import datetime


def validar_nombre(texto):
    if not texto.strip():
        raise ValueError("El nombre no puede estar vacio.")
    if not re.match(r"^[A-Za-záéíóúÁÉÍÓÚñÑ' ]+$", texto):
        raise ValueError("El nombre solo puede contener letras y espacios.")
    return texto.strip().title()


def validar_email(email):
    if not email.strip():
        raise ValueError("El email no puede estar vacio.")
    if not re.match(r"^[\w\.-]+@[\w\.-]+\.\w+$", email):
        raise ValueError("El email no tiene un formato valido (ej: usuario@correo.com).")
    return email.strip().lower()


def validar_password(password):
    if len(password) < 4:
        raise ValueError("La contrasena debe tener al menos 4 caracteres.")
    return password


def validar_fecha(fecha):
    try:
        datetime.strptime(fecha, "%d/%m/%Y")
    except ValueError:
        raise ValueError("La fecha debe tener formato DD/MM/AAAA con una fecha real.")
    return fecha


def validar_hora(hora):
    try:
        datetime.strptime(hora, "%H:%M")
    except ValueError:
        raise ValueError("La hora debe tener formato HH:MM (ej: 08:00, 20:00).")
    return hora


def validar_dosis(dosis):
    if not dosis.strip():
        raise ValueError("La dosis no puede estar vacia.")
    return dosis.strip()


def validar_celular(texto):
    if not texto.strip():
        raise ValueError("El telefono no puede estar vacio.")
    if not re.match(r"^\+?[\d\- ]{8,15}$", texto):
        raise ValueError("El telefono debe tener solo digitos (ej: +505 86459285).")
    return texto.strip()


def validar_cantidad(cantidad):
    if not cantidad.strip():
        raise ValueError("La cantidad no puede estar vacia.")
    if not cantidad.isdigit():
        raise ValueError("La cantidad debe ser un numero entero positivo.")
    return int(cantidad)


def validar_descripcion(texto):
    if not texto.strip():
        raise ValueError("El dato no puede estar vacio.")
    return texto.strip().capitalize()


def pedir_campo(etiqueta, validador):
    while True:
        try:
            valor = input(etiqueta)
            return validador(valor)
        except ValueError as e:
            print(f"  [Error] {e}")
```

Codigo 4 - Operaciones CRUD (ejemplo Usuario)

4) Operaciones CRUD (ej. Usuario)

Crear, buscar, listar, actualizar, eliminar (Funciones/usuario.py) | Funciones/usuario.py (líneas 1-70)

```python
from Entidades import Usuario
from Datos import usuarios
from .validaciones import validar_nombre, validar_email, validar_password, validar_celular, pedir_campo


def registrar_usuario():
    print("\n--- Registrar Usuario ---")
    nombre = pedir_campo("Nombre completo: ", validar_nombre)
    email = pedir_campo("Email: ", validar_email)
    if buscar_usuario(email):
        print("  [Error] Ya existe un usuario con ese email.")
        return
    password = pedir_campo("Contrasena: ", validar_password)
    telefono = pedir_campo("Telefono: ", validar_celular)
    nuevo_id = max([u.id for u in usuarios], default=0) + 1
    usuarios.append(Usuario(id=nuevo_id, nombre=nombre, email=email,
                            password=password, telefono=telefono))
    print(f"  Usuario registrado con exito. ID: {nuevo_id}")


def buscar_usuario(email):
    for u in usuarios:
        if u.email == email:
            return u
    return None


def listar_usuarios():
    if not usuarios:
        print("  No hay usuarios registrados.")
        return
    for u in usuarios:
        print(f"  ID:{u.id} | {u.nombre} | {u.email} | {u.telefono}")


def actualizar_usuario(email):
    u = buscar_usuario(email)
    if not u:
        print("  [Error] No se encontro un usuario con ese email.")
        return
    print(f"  Editando: {u.nombre} (ID: {u.id})")
    print("  Deje el campo vacio para no cambiar el dato.")
    nombre = input("Nuevo nombre: ")
    if nombre.strip():
        u.nombre = validar_nombre(nombre)
    telefono = input("Nuevo telefono: ")
    if telefono.strip():
        u.telefono = validar_celular(telefono)
    print("  Usuario actualizado con exito.")


def eliminar_usuario(email):
    u = buscar_usuario(email)
    if not u:
        print("  [Error] No se encontro un usuario con ese email.")
        return
    usuarios.remove(u)
    print("  Usuario eliminado con exito.")


def contar_usuarios():
    return len(usuarios)
```

Codigo 5 - Busqueda lineal (recorrido elemento por elemento)

5) Algoritmo de Búsqueda Lineal

Recorrido secuencial elemento por elemento | Funciones/busqueda_lineal.py (líneas 1-60)

```python
def busqueda_lineal(lista, criterio, atributo):
    """
    Implementacion de BUSQUEDA LINEAL
    Recorre elemento por elemento hasta encontrar coincidencia
    """
    resultados = []
    if not lista:
        return resultados

    for elemento in lista:
        valor = getattr(elemento, atributo, None)
        if valor is None:
            continue

        # Comparacion por igualdad (case insensitive para strings)
        if isinstance(valor, str) and isinstance(criterio, str):
            if valor.lower() == criterio.lower():
                resultados.append(elemento)
        elif valor == criterio:
            resultados.append(elemento)

    return resultados


def busqueda_lineal_por_email(usuarios, email):
    """Busqueda lineal especifica para usuarios por email"""
    for u in usuarios:
        if u.email.lower() == email.lower():
            return u
    return None


def busqueda_lineal_citas_por_medico(citas, medico_id):
    """Busqueda lineal para filtrar citas por medico_id"""
    encontrados = []
    for c in citas:
        if c.medico_id == medico_id:
            encontrados.append(c)
    return encontrados


def busqueda_lineal_medicamento_por_nombre(medicamentos, nombre):
    """Busqueda lineal para filtrar medicamentos por nombre"""
    encontrados = []
    for m in medicamentos:
        if m.nombre.lower() == nombre.lower():
            encontrados.append(m)
    return encontrados
```

Codigo 6 - Cola con prioridad (heapq): adulto mayor primero

6) Cola con Prioridad (heapq)

Adulto mayor prioridad 0, normal 1, turno_global (ordenamiento) | Funciones/cola_atencion.py (líneas 1-56)

```python
import heapq


class ColaAtencion:
    """Implementacion de una Cola CON PRIORIDAD para gestionar la atencion de citas
    Prioridad: 1 = Adulto Mayor (ATENCION PRIORITARIA) | 2 = Normal
    """
    def __init__(self):
        self.cola = []
        self.contador = 0
        self.turno_global = 0

    def encolar(self, item, es_adulto_mayor=False):
        """Agregar elemento a la cola con prioridad"""
        self.contador += 1
        self.turno_global += 1
        # Prioridad: 0 = Mayor prioridad (Adulto Mayor), 1 = Normal
        if es_adulto_mayor:
            prioridad = 0
            tipo = "ADULTO MAYOR (PRIORITARIO)"
        else:
            prioridad = 1
            tipo = "NORMAL"
        item['turno'] = self.turno_global
        item['prioridad'] = tipo
        # (prioridad, turno_global, item) - heapq usa el primer elemento para ordenar
        heapq.heappush(self.cola, (prioridad, self.turno_global, item))
        print(f"  [COLA] Paciente {item.get('paciente', '')} agregado. Turno #{self.turno_global} | {tipo}")

    def desencolar(self):
        """Atender (eliminar) elemento con MAYOR PRIORIDAD"""
        if self.esta_vacia():
            print("  [COLA] No hay pacientes en espera para atender.")
            return None
        prioridad, turno, atendido = heapq.heappop(self.cola)
        print(f"  [COLA] ATENDIENDO -> {atendido.get('prioridad')} | Paciente: {atendido.get('paciente', '')}
| Cita ID: {atendido.get('cita_id', '')} | Turno: {atendido.get('turno')}")
        return atendido

    def mostrar_cola(self):
        """Mostrar todos los pacientes en espera con su prioridad"""
        if self.esta_vacia():
            print("  [COLA] La cola de atencion esta vacia.")
            return
        print("\n--- COLA DE ATENCION CON PRIORIDAD ---")
        print("  (Adulto Mayor = Prioritario | Se atiende primero sin importar orden de llegada)")
        # Copiar y ordenar para mostrar sin alterar la cola
        temp = sorted(self.cola, key=lambda x: (x[0], x[1]))
        for i, (prio, turno, item) in enumerate(temp, 1):
            print(f"  {i}. [{item.get('prioridad')}] Turno: {item.get('turno')} | Paciente:
{item.get('paciente')} | Cita ID: {item.get('cita_id')} | Hora: {item.get('hora')}")
        print(f"\n  Total en espera: {self.tamanio()}")

    def tamanio(self):
        return len(self.cola)

    def esta_vacia(self):
        return len(self.cola) == 0
```

Codigo 7 - Interfaz Tkinter (app/main.py)

7) Interfaz Tkinter - App principal

Carga de datos (Funciones.sistema) y navegación de pantallas | app/main.py (líneas 1-58)

```python
import tkinter as tk
import sys
import os

ROOT_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
if ROOT_DIR not in sys.path:
    sys.path.append(ROOT_DIR)

# Precarga datos usando la estructura requerida (Funciones/Datos/Entidades)
try:
    from Funciones import sistema
    sistema.iniciar_sistema()
except Exception:
    pass

from screens.splash_screen import SplashScreen
from screens.inicio_screen import InicioScreen
from screens.login_screen import LoginScreen
from screens.registro_screen import RegistroScreen
from screens.home_screen import HomeScreen
from screens.citas_screen import CitasScreen
from screens.historial_screen import HistorialScreen
from screens.medicamentos_screen import MedicamentosScreen
from screens.estadisticas_screen import EstadisticasScreen
from screens.atencion_screen import AtencionScreen
from screens.forms.form_medicamento import FormMedicamento
from screens.forms.form_registro import FormRegistro
from screens.password_reset.screen_2_otp import Screen2OTP

class SalunicApp(tk.Tk):
    def __init__(self):
        tk.Tk.__init__(self)

        self.title("SALUNIC - Salud y Bienestar")
        self.geometry("375x812")
        self.resizable(False, False)

        self.current_user = None

        container = tk.Frame(self)
        container.pack(side="top", fill="both", expand=True)
        container.grid_rowconfigure(0, weight=1)
        container.grid_columnconfigure(0, weight=1)

        self.frames = {}
        self.container = container

        self.show_screen("Splash")

    def show_screen(self, name):
        if name in self.frames:
            frame = self.frames[name]
            frame.tkraise()
        else:
            if name == "Splash":
                frame = SplashScreen(self.container, self)
            elif name == "Inicio":
                frame = InicioScreen(self.container, self)
```

Codigo 8 - Consola integrada (Entidades/Datos/Funciones)

8) Módulo Consola integrado

Usa Entidades, Datos y Funciones (menú, cola con prioridad, JSON) | consola/main.py (líneas 1-52)

```python
import sys
import os

ROOT_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
if ROOT_DIR not in sys.path:
    sys.path.insert(0, ROOT_DIR)

# Estructura requerida por el profesor: consola usa Entidades, Datos y Funciones
from Datos import usuarios, citas, medicamentos, almacen
from Datos.datos import cargar_datos, guardar_datos, cargar_datos_iniciales
from Funciones import usuario as f_usuario
from Funciones import cita_medica as f_cita
from Funciones import medicamento as f_medicamento
from Funciones import almacen_medicamento as f_almacen
from Funciones.cola_atencion import cola_atencion

TITULO = "SALUNIC - Sistema de Salud y Bienestar"
LINEA = "=" * 55
SUB = "-" * 55


def cabecera_modulo(nombre):
    print("\n" + SUB)
    print(f"  MENU {nombre}")
    print(SUB)


def pedir_opcion():
    while True:
        try:
            opc = input("Seleccione una opcion: ").strip()
            if not opc.isdigit():
                raise ValueError("Debe ingresar un numero.")
            return int(opc)
        except ValueError as e:
            print(f"  [Error] {e}")


def mostrar_menu_principal():
    print("\n" + LINEA)
    print(TITULO)
    print(LINEA)
    print("SELECCIONE UN MODULO:")
    print("1. Gestion de Usuarios")
    print("2. Gestion de Citas Medicas")
    print("3. Gestion de Medicamentos")
    print("4. Gestion de Almacen / Inventario")
    print("5. Cola de Atencion con Prioridad")
    print("6. Archivo de Datos (Guardar/Cargar)")
    print(SUB)
    print("0. Salir")

```